

Explicación de la estructura del proyecto

SISTEMA POS

La estructura está dividida en dos aplicaciones principales: **backend**, que contiene la API central y las reglas reales del negocio, y **client**, que contiene la aplicación de escritorio instalada en cada computadora. Esta separación es importante porque las computadoras de los usuarios no deben acceder directamente a MySQL. El cliente se comunica con la API mediante HTTPS y JSON; la API es la única que autentica, autoriza, valida reglas, ejecuta transacciones y accede a la base de datos.

La idea fundamental es la siguiente:

El frontend muestra y solicita; Wails conecta; la API decide; el dominio define las reglas; la infraestructura persiste y comunica con servicios externos.

1. Directorio raíz

Elemento	Propósito
backend/	Código del servidor, API, casos de uso, dominio y persistencia. Se ejecuta en el servidor central.
client/	Aplicación de escritorio Wails instalada en las PCs de la empleada y de la administradora.

No conviene mezclar el código del backend con el código del cliente. Aunque ambos estén en el mismo repositorio, tienen responsabilidades, ciclos de compilación y riesgos de seguridad distintos.

2. Backend

El backend es el núcleo confiable del sistema. Todas las operaciones importantes deben terminar aquí, incluso cuando el cliente ya haya realizado validaciones visuales preliminares.

Plain Text

```
backend/  
├─ cmd/  
├─ internal/  
├─ db/  
├─ contracts/  
├─ Dockerfile  
├─ docker-compose.yml  
├─ go.mod  
└─ go.sum
```

2.1 backend/cmd/

`cmd` contiene los puntos de entrada ejecutables de la aplicación. En este caso, la API es el ejecutable principal.

backend/cmd/api/

Este directorio contiene el programa que arranca el servidor HTTP de la API.

backend/cmd/api/main.go

`main.go` debe ser deliberadamente pequeño. Su función no es contener reglas de ventas o inventario, sino construir y conectar las dependencias de la aplicación.

Normalmente realiza estas tareas:

1. Lee la configuración del entorno.
2. Inicializa el logger.
3. Abre la conexión a MySQL.
4. Inicializa el gestor de transacciones.
5. Construye repositorios y adaptadores.
6. Construye los casos de uso.
7. Registra las rutas HTTP.
8. Configura TLS, timeouts y apagado controlado.
9. Inicia el servidor.

Conceptualmente, `main.go` funciona como el **composition root**: el lugar donde se decide qué implementación concreta satisface cada interfaz.

No debe contener algo como `if stock < 0` o `if user.Role == "admin"`. Esas son reglas de negocio o autorización y deben vivir en las capas correspondientes.

3. backend/internal/

En Go, un paquete dentro de `internal` solo puede ser importado por código que se encuentre dentro del árbol padre permitido. Esto ayuda a evitar que otros proyectos importen directamente detalles internos de la API.

Aquí vive la mayor parte del código privado del backend. La estructura se divide en:

Plain Text

```
internal/  
├─ platform/  
├─ modules/  
└─ shared/
```

3.1 backend/internal/platform/

`platform` contiene capacidades técnicas transversales. Son componentes necesarios para ejecutar la aplicación, pero no representan reglas específicas de Provisiones Gurima.

Una regla útil es esta:

`platform` puede ofrecer infraestructura a los módulos, pero no debe decidir cómo funciona una venta, una devolución o un cierre de caja.

platform/config/

Centraliza la lectura y validación de configuración de la aplicación.

Puede manejar:

Configuración	Ejemplo
Servidor	Puerto HTTP, timeouts y límites de solicitudes
Base de datos	Host, puerto, nombre, usuario y configuración del pool
Seguridad	Claves de sesión, duración de tokens y configuración TLS
Correo	Servidor SMTP, remitente y destinatario administrativo
Entorno	Desarrollo, pruebas o producción

La configuración debe provenir del entorno o de archivos seguros. No deben colocarse contraseñas de MySQL ni secretos dentro del repositorio.

platform/httpserver/

Contiene la configuración técnica del servidor HTTP.

Aquí pueden vivir:

- Creación del servidor `net/http` .
- Registro de rutas.
- Middleware de recuperación ante errores inesperados.
- Middleware de logging de solicitudes.
- Middleware de autenticación de sesión.
- CORS, si fuera necesario.
- Límites de tamaño de solicitudes.
- Timeouts y apagado ordenado.

Este directorio no debe decidir si una empleada puede aprobar una devolución. Solo debe transportar la solicitud hacia el caso de uso, mientras que la autorización concreta pertenece a la aplicación.

platform/auth/

Contiene la implementación técnica de autenticación y elementos comunes de identidad.

Puede incluir:

- Hash y verificación de contraseñas.
- Creación y validación de sesiones.
- Extracción del usuario autenticado desde el contexto de la solicitud.
- Carga de permisos técnicos.
- Expiración y revocación de sesiones.

Debe distinguirse **autenticación** de **autorización**. Autenticación responde “¿quién es este usuario?” . Autorización responde “¿puede ejecutar este caso de uso?” . La primera puede apoyarse en `platform/auth` ; la segunda debe ser evaluada por las políticas de aplicación y los casos de uso.

platform/mysql/

Contiene la inicialización técnica de la conexión con MySQL.

Puede encargarse de:

- Abrir el `*sql.DB`.
- Configurar el pool de conexiones.
- Verificar la conectividad.
- Configurar límites de conexiones e idle time.
- Cerrar correctamente la conexión durante el apagado.

No debe contener consultas de ventas, inventario o crédito. Las consultas específicas deben vivir en la infraestructura del módulo correspondiente.

platform/transaction/

Abstrae la ejecución de transacciones de base de datos.

Es necesaria porque una operación como completar una venta puede modificar varias áreas al mismo tiempo:

Plain Text

venta + pagos + caja + inventario + crédito + auditoría

Si una parte falla, la operación completa debe revertirse. Este directorio puede definir un `TransactionManager`, un `UnitOfWork` o un mecanismo equivalente para ejecutar varios repositorios dentro de la misma transacción.

La transacción no debe esconder reglas de negocio. Su responsabilidad es garantizar la unidad técnica de persistencia; el caso de uso decide qué operaciones deben formar parte de ella.

platform/clock/

Proporciona la hora actual mediante una abstracción, por ejemplo:

Go

```
type Clock interface {  
    Now( ) time.Time  
}
```

Esto evita llamar directamente a `time.Now()` dentro de toda la lógica de negocio. En producción se usa un reloj real; en las pruebas se usa un reloj controlado.

Es especialmente útil para probar vencimientos de crédito, sesiones de caja, fechas de documentos, alertas de productos vencidos y cierres diarios.

platform/id/

Centraliza la generación de identificadores técnicos, como UUIDs u otros identificadores elegidos para el sistema.

Una abstracción de identificadores permite utilizar valores deterministas en las pruebas y evita que cada módulo implemente su propia estrategia.

platform/mail/

Contiene el adaptador técnico para enviar correos electrónicos.

Puede utilizarse para enviar a la administradora:

- Resumen de alertas de inventario.
- Productos próximos a vencer.
- Morosidad.
- Confirmaciones operativas.
- Notificaciones de errores o respaldos.

El módulo `reporting` o `operations` puede solicitar el envío a través de un puerto, pero no debe conocer SMTP ni construir conexiones de correo directamente.

platform/backup/

Contiene la integración técnica con el mecanismo de respaldos y restauración.

Puede incluir:

- Ejecución del backup de MySQL.
- Compresión y almacenamiento de archivos.
- Validación de existencia e integridad del respaldo.
- Restauración controlada.
- Retención y limpieza de copias antiguas.

La restauración debe requerir autorización administrativa y registrar auditoría. Este directorio no debe decidir qué usuario tiene permiso: implementa la capacidad técnica; el caso de uso `RestoreBackup` aplica la política de seguridad.

platform/logging/

Configura el logging técnico estructurado, compatible con `log/slog`.

Los logs pueden registrar:

- Inicio y apagado de la API.
- Duración de solicitudes.
- Errores inesperados.
- Fallos de conexión.
- Identificadores de correlación.

El log técnico no sustituye la auditoría de negocio. Un mensaje como “request failed” no es equivalente a una entrada que indique que una administradora aprobó una devolución de RD\$1,500.

4. backend/internal/modules/

Aquí viven los módulos de negocio. Cada módulo agrupa una capacidad funcional completa y mantiene internamente sus capas de Clean Architecture.

Plain Text

```
modules/  
├─ identity/  
├─ catalog/  
├─ sales/  
├─ cash/  
├─ inventory/  
├─ purchasing/  
├─ credit/  
├─ returns/  
├─ billing/  
├─ reporting/  
├─ audit/  
└─ operations/
```

La razón de organizar el código por módulo es evitar que el proyecto termine dividido únicamente por tipos técnicos. Una estructura como:

Plain Text

```
controllers/  
services/  
models/  
repositories/
```

mezcla ventas, caja, inventario y crédito dentro de las mismas carpetas. Con el tiempo, esto dificulta saber qué código pertenece a cada regla de negocio.

En cambio, en la estructura propuesta, todo lo relacionado con ventas comienza en `modules/sales`, todo lo relacionado con inventario en `modules/inventory`, y así sucesivamente.

Cada módulo puede tener estas cuatro capas:

Carpeta interna	Función
<code>domain/</code>	Entidades, objetos de valor, invariantes y errores propios del módulo
<code>application/</code>	Casos de uso, comandos, consultas y puertos
<code>adapters/</code>	Entrada HTTP, DTOs, JSON, presenters y traducción de errores
<code>infrastructure/</code>	Implementaciones concretas de persistencia y servicios externos

4.1 `modules/identity/`

Gestiona la identidad y el acceso al sistema.

Puede contener:

- Usuarios.
- Roles.
- Permisos.
- Sesiones.
- Cambio y recuperación de credenciales.
- Activación o desactivación de usuarios.

El rol de la empleada y el rol de la propietaria se modelan aquí, pero las reglas contextuales de cada operación —por ejemplo, quién puede aprobar un ajuste— deben verificarse también en el caso de uso concreto.

identity/domain/

Define conceptos como `User` , `Role` , `Permission` y `Session` , además de sus invariantes. Por ejemplo, un usuario desactivado no debe iniciar una nueva sesión.

identity/application/

Coordina casos de uso como `Login` , `Logout` , `CreateUser` , `ChangeUserRole` y `ListPermissions` .

identity/application/commands/

Contiene operaciones que modifican estado, como crear un usuario, cambiar una contraseña, desactivar una cuenta o revocar una sesión.

identity/application/queries/

Contiene operaciones de lectura, como buscar usuarios, consultar roles o verificar información de una sesión.

identity/application/ports/

Define interfaces que el caso de uso necesita, como un repositorio de usuarios, un proveedor de hash, un almacén de sesiones o un reloj.

identity/adapters/http/

Expone los endpoints HTTP de autenticación y administración de usuarios. Convierte JSON en comandos, invoca el caso de uso y transforma el resultado en una respuesta HTTP.

El handler no debe implementar la lógica de autenticación completa ni acceder directamente a MySQL.

identity/infrastructure/mysql/

Implementa los repositorios de identidad utilizando SQL y el código generado por `sqlc` .

4.2 modules/catalog/

Gestiona el catálogo de productos, categorías, unidades, códigos de barras, costos, precios, mínimos y estados activo/inactivo.

El catálogo es utilizado por ventas, compras e inventario, pero esos módulos no deben modificar directamente sus tablas. Deben utilizar interfaces o casos de uso bien definidos.

Ejemplos de casos de uso:

- Crear producto.
 - Actualizar precio.
 - Desactivar producto.
 - Buscar producto por código.
 - Consultar productos próximos a mínimo.
-

4.3 `modules/sales/`

Representa el proceso de venta del POS.

Puede encargarse de:

- Crear una venta.
- Agregar y quitar líneas.
- Calcular subtotal y total.
- Aplicar descuentos dentro del límite.
- Completar la venta.
- Registrar pagos mixtos.
- Asociar un cliente.
- Generar una venta pendiente o apartada, si forma parte del alcance final.

La venta es uno de los módulos más importantes porque coordina con caja, inventario, crédito, facturación y auditoría.

El caso de uso `CompleteSale` debe ejecutarse en una transacción y debe volver a calcular en el servidor los precios, descuentos, impuestos y totales recibidos desde el cliente.

4.4 `modules/cash/`

Gestiona la caja física y sus sesiones o turnos.

Puede incluir:

- Apertura de caja.
- Fondo inicial.
- Movimientos de efectivo.
- Retiros.
- Ventas por medio de pago.

- Cierre de caja.
- Efectivo esperado y efectivo contado.
- Faltantes y sobrantes.
- Revisión y reapertura autorizada.

cash debe conservar la diferencia entre el valor esperado y el valor contado. No se debe corregir el saldo silenciosamente; una diferencia debe quedar como movimiento o resultado de cierre con su motivo y aprobación.

4.5 modules/inventory/

Gestiona existencias y movimientos de inventario.

Puede encargarse de:

- Existencias actuales.
- Entradas por compra.
- Salidas por venta.
- Conteos físicos.
- Ajustes.
- Mermas.
- Lotes y vencimientos.
- Mínimos y alertas.
- Existencias negativas controladas.

La regla principal es que una modificación de inventario debe generar un movimiento trazable. No se debe actualizar solamente una columna stock sin registrar por qué cambió.

Una existencia negativa puede permitirse por decisión operativa, pero debe generar advertencia y quedar reportable. Esto no debe convertirse en una operación invisible.

4.6 modules/purchasing/

Gestiona proveedores, compras y cuentas por pagar relacionadas con las compras.

Puede incluir:

- Alta y consulta de proveedores.
- Creación de compras.

- Recepción completa o parcial.
- Costos de adquisición.
- Lotes y fechas de vencimiento.
- Incidencias de recepción.
- Historial de compras.
- Obligaciones con proveedores.

La recepción parcial es una regla importante: si se ordenaron 20 unidades y se reciben 15, el inventario aumenta en 15, mientras que las 5 restantes quedan como diferencia o incidencia. El caso de uso debe aplicar esa regla, no la pantalla ni una consulta aislada.

4.7 modules/credit/

Gestiona el fiado o crédito otorgado a clientes.

Puede incluir:

- Clientes.
- Límite de crédito.
- Plazo de pago.
- Ventas a crédito.
- Abonos.
- Saldos.
- Vencimientos.
- Morosidad.
- Bloqueo de crédito.
- Autorizaciones excepcionales.

El saldo debe poder explicarse mediante cargos y abonos. Es preferible conservar un libro de movimientos de crédito que modificar simplemente el saldo sin historial.

Una venta a crédito debe validar el límite, el vencimiento y el bloqueo antes de completarse, salvo que exista una autorización válida y auditable.

4.8 modules/returns/

Gestiona anulaciones, devoluciones, cambios, reembolsos y saldos a favor.

Este módulo debe aplicar reglas como:

- Motivo obligatorio.
- Autorización para operaciones sensibles.
- Reembolso mediante un medio permitido.
- Reintegro al inventario solo si el producto es vendible.
- Registro como merma cuando el producto no puede volver a venderse.
- Emisión de nota de crédito cuando corresponda.

No se debe borrar la venta original. La devolución debe referenciarla y producir movimientos compensatorios.

4.9 modules/billing/

Gestiona los comprobantes y la información fiscal.

Puede contener:

- Tipos de comprobante.
- Numeración.
- Impuestos parametrizables.
- NCF/e-NCF, según la decisión autorizada.
- Notas de crédito.
- Presentación digital del comprobante.

Este módulo debe diseñarse con un puerto para un proveedor fiscal externo. Así se puede comenzar con comprobantes configurables y añadir una integración avanzada posteriormente sin introducir esa dependencia en ventas o en el frontend.

La lógica fiscal debe estar centralizada y versionada. No debe calcularse de forma independiente en cada pantalla.

4.10 modules/reporting/

Gestiona consultas de lectura, informes, filtros, agregaciones, exportaciones y alertas.

Este módulo no debe modificar operaciones de negocio. Puede tener consultas optimizadas directamente para reportes, conocidas como **read models**.

Ejemplos:

- Ventas por fecha.

- Resumen de caja.
- Existencias bajo mínimo.
- Productos próximos a vencer.
- Compras por proveedor.
- Cuentas por cobrar.
- Cuentas por pagar.
- Mermas.
- Diferencias de inventario.

Las consultas pueden retornar modelos específicos para cada reporte. No es necesario reutilizar una entidad de dominio compleja para mostrar una tabla de ventas.

4.11 modules/audit/

Gestiona la auditoría de acciones sensibles.

Una entrada de auditoría puede registrar:

Campo	Ejemplo
Actor	Usuario que ejecutó la operación
Rol	Empleada o administradora
Acción	APPROVE_INVENTORY_ADJUSTMENT
Entidad	Ajuste de inventario
Identificador	ID del ajuste
Fecha	Momento de la operación
Motivo	Razón declarada por el usuario
Autorización	Usuario que aprobó, si aplica
Valores	Resumen anterior y nuevo

La auditoría debe ser append-only: se agregan entradas, pero no se editan ni eliminan mediante las operaciones normales del sistema.

Los módulos pueden solicitar el registro de una auditoría mediante un puerto. No deben insertar directamente en la tabla de auditoría desde cada handler.

4.12 modules/operations/

Gestiona operaciones administrativas y técnicas del sistema.

Puede incluir:

- Migración de datos históricos.
- Carga del inventario inicial.
- Validación de datos importados.
- Ejecución de respaldos.
- Restauración autorizada.
- Tareas de mantenimiento.
- Reprocesamiento seguro de operaciones fallidas.

Una migración histórica no debe insertar datos sin control. Debe tener estados como `draft` , `validated` , `approved` y `applied` , de modo que la administradora pueda revisar y aprobar la carga antes de activarla en producción.

5. backend/internal/shared/

`shared` contiene piezas realmente comunes y estables. Debe utilizarse con moderación. Si se coloca aquí cualquier cosa que dos módulos usen, se termina creando un “cajón de sastre” con dependencias globales.

La regla recomendada es:

Algo pertenece a `shared` solo si tiene el mismo significado de negocio en varios módulos y no pertenece claramente a un módulo específico.

5.1 shared/domain/

Contiene conceptos transversales del dominio.

`shared/domain/money/`

Define el objeto de valor para representar dinero.

Debe evitar `float32` y `float64` . Puede utilizar unidades menores con `int64` , moneda explícita y operaciones de suma, resta, multiplicación y redondeo controlado.

Ejemplo conceptual:

Go

```
type Money struct {  
    AmountMinor int64  
    Currency    Currency  
}
```

Así, la lógica de ventas, caja, crédito y compras utiliza el mismo comportamiento para importes en DOP.

shared/domain/pagination/

Define estructuras comunes para consultas paginadas, como página, tamaño, total y cursor si se utiliza ese enfoque.

Debe contener solamente la abstracción de paginación, no consultas específicas de productos o ventas.

shared/domain/errors/

Define errores de negocio comunes y una forma consistente de clasificarlos.

Por ejemplo:

- Producto no encontrado.
- Operación no permitida.
- Caja no abierta.
- Crédito bloqueado.
- Estado inválido.
- Conflicto de idempotencia.
- Datos inválidos.

Los adaptadores HTTP pueden convertir estos errores a códigos HTTP y respuestas JSON sin que el dominio conozca HTTP.

5.2 shared/application/authorization/

Contiene políticas y tipos comunes para autorización de casos de uso.

Puede definir conceptos como:

- Permiso requerido.
- Usuario autenticado.

- Contexto de autorización.
- Aprobador.
- Motivo de autorización.
- Regla para operaciones sensibles.

Debe evitar convertirse en un sistema genérico desconectado del negocio. La autorización común puede vivir aquí, pero la decisión contextual —por ejemplo, si una devolución requiere reembolso aprobado— pertenece al caso de uso de devoluciones.

6. backend/db/

Este directorio contiene la definición y la implementación técnica de la persistencia SQL.

Plain Text

```
db/  
├─ migrations/  
├─ queries/  
└─ generated/  
    └─ sqlc/
```

6.1 db/migrations/

Contiene migraciones versionadas para crear y modificar el esquema de MySQL.

Cada migración debe tener una parte `up` y, cuando sea viable, una parte `down`.

Ejemplos de cambios:

- Crear tabla de productos.
- Crear tabla de ventas.
- Añadir el estado de una devolución.
- Crear un índice para reportes.
- Añadir el identificador de idempotencia.

Las migraciones deben ejecutarse con `golang-migrate`. No se debe modificar manualmente el esquema productivo sin una migración versionada, porque el entorno de desarrollo, pruebas y producción podrían quedar diferentes.

6.2 db/queries/

Contiene los archivos SQL escritos por el equipo para que `sqlc` genere código Go tipado.

Aquí se colocan consultas como:

- Buscar un producto por código.
- Obtener una venta y sus líneas.
- Insertar una entrada de inventario.
- Obtener el saldo de un cliente.
- Consultar el resumen de caja.

Las consultas deben agruparse de manera comprensible, por ejemplo por módulo o capacidad. `sqlc` genera código desde estas consultas, pero SQL no debe convertirse en el lugar donde se esconden todas las reglas de negocio.

6.3 `db/generated/`

Contiene código generado automáticamente. No se debe editar manualmente.

`db/generated/sqlc/`

Aquí se genera el código Go a partir de los SQL de `db/queries`.

Este código conoce la forma de las tablas y de las filas, por lo que pertenece a persistencia, no al dominio. Los repositorios de cada módulo deben transformar esos tipos generados a entidades de dominio o modelos de aplicación.

Por ejemplo:

Plain Text

```
sqlc.ProductRow
  | mapeo
  ▼
catalog.Product
```

El dominio no debe importar `db/generated/sqlc`. Si se cambia una columna o se reorganiza una tabla, el impacto debe quedarse principalmente en la infraestructura.

7. `backend/contracts/`

Contiene los contratos externos del backend, especialmente los contratos HTTP que consumen los clientes.

7.1 contracts/http/

Puede contener documentación o definiciones de:

- Rutas.
- Métodos HTTP.
- Formatos JSON de entrada.
- Formatos JSON de salida.
- Códigos de error.
- Reglas de paginación.
- Requisitos de idempotencia.

Ejemplo conceptual:

Plain Text

```
POST /api/v1/sales
```

Request:

```
{
  "cash_session_id": "...",
  "items": [...],
  "payments": [...],
  "idempotency_key": "..."
}
```

Los contratos sirven para que el cliente y la API evolucionen coordinadamente. No deben contener entidades internas de dominio ni tipos específicos de `sqlc`.

Como el stack aprobado no incluye Swagger u OpenAPI, estos contratos pueden mantenerse inicialmente como Markdown, JSON Schema manual o documentación versionada. Lo importante es que las respuestas y errores sean explícitos y estables.

8. Archivos de infraestructura del backend

8.1 backend/Dockerfile

Define cómo construir la imagen del backend para despliegue.

Puede incluir:

- Imagen base de compilación de Go.

- Descarga de dependencias.
- Compilación del binario.
- Imagen final más pequeña.
- Usuario no privilegiado.
- Certificados necesarios.
- Comando de arranque.

La imagen no debe incluir secretos. Las credenciales y la configuración deben inyectarse en el entorno de ejecución.

8.2 backend/docker-compose.yml

Describe el entorno local de desarrollo, especialmente el contenedor MySQL y, si se decide, el servicio de la API.

Puede definir:

- MySQL.
- Volúmenes locales.
- Variables de entorno de desarrollo.
- Red interna.
- Health checks.
- Dependencias de inicio.

Docker Compose sirve para reproducir el entorno de desarrollo. No necesariamente es el mecanismo final de producción.

8.3 backend/go.mod

Declara el módulo Go y sus dependencias directas.

Debe incluir las dependencias autorizadas del backend, como el driver de MySQL y las herramientas o librerías necesarias para migraciones y pruebas.

El nombre del módulo debe ser estable, porque otros paquetes del backend lo utilizarán en sus imports.

8.4 backend/go.sum

Contiene hashes de verificación de las dependencias Go. Se genera y actualiza mediante las herramientas de Go; no debe editarse manualmente salvo casos excepcionales.

Debe versionarse junto con `go.mod` para que las compilaciones sean reproducibles.

9. Cliente de escritorio Wails

El cliente es la aplicación instalada en cada PC. Su responsabilidad es presentar la interfaz y comunicarse con la API. No es la autoridad del negocio.

Plain Text

```
client/  
├─ frontend/  
├─ app.go  
├─ api_client.go  
└─ wails.json
```

9.1 `client/frontend/`

Contiene el proyecto Svelte, TypeScript, Vite y Tailwind CSS.

`client/frontend/src/`

Es el código fuente de la interfaz.

`src/app/`

Contiene la composición general de la aplicación:

- Inicialización.
- Layout raíz.
- Tema visual.
- Estado global de sesión.
- Manejo general de errores.
- Providers o configuración común.

No debe contener la lógica detallada de una venta o de un cierre de caja.

`src/routes/`

Contiene la navegación y las páginas principales de la aplicación.

Puede incluir rutas como:

Plain Text

```
/login  
/pos  
/cash  
/inventory  
/purchases  
/credit  
/reports  
/settings
```

Una ruta debe coordinar la pantalla y sus componentes. No debe ejecutar SQL ni decidir reglas de autorización.

src/features/

Agrupar el frontend por capacidades de negocio.

Esta organización permite que los componentes, stores y servicios relacionados con una funcionalidad estén juntos.

src/features/pos/

Incluye:

- Búsqueda y selección de productos.
- Carrito.
- Líneas de venta.
- Descuentos visibles.
- Selección de medios de pago.
- Pagos mixtos.
- Cálculo visual del cambio.
- Confirmación de venta.

El backend debe recalcular el total definitivo. El cálculo del frontend es una ayuda de interacción, no una fuente de verdad.

src/features/cash/

Incluye las pantallas de apertura, movimientos, cierre, diferencias y solicitud de reapertura.

src/features/inventory/

Incluye existencias, conteos, ajustes, mermas, vencimientos y alertas.

src/features/purchases/

Incluye proveedores, compras, recepción parcial, costos, lotes y cuentas por pagar.

src/features/credit/

Incluye clientes, saldos, abonos, vencimientos, morosidad y estado del crédito.

src/features/reports/

Incluye filtros, tablas, exportaciones y estados de carga para los reportes.

src/components/

Contiene componentes visuales reutilizables que no pertenecen exclusivamente a un módulo.

Ejemplos:

- Botón.
- Modal.
- Tabla.
- Selector.
- Campo de moneda.
- Date picker.
- Mensaje de error.
- Indicador de carga.

Si un componente conoce reglas específicas de crédito o inventario, probablemente debe vivir en el `feature` correspondiente y no en `components`.

src/stores/

Contiene el estado visual y de sesión de la aplicación.

Puede almacenar:

- Usuario actual.
- Estado de autenticación.
- Caja activa mostrada en la interfaz.
- Carrito actual.
- Filtros seleccionados.
- Estado de carga.

- Errores de presentación.

Un store no debe ser la autoridad para aprobar un descuento o decidir si existe inventario suficiente. Esas decisiones pertenecen al backend.

src/services/

Contiene servicios de frontend que no son componentes visuales.

src/services/api/

Centraliza la comunicación con la API a través del puente Wails o del cliente HTTP utilizado por la aplicación.

Puede manejar:

- Construcción de solicitudes.
- Serialización JSON.
- Sesión.
- Headers.
- Timeouts.
- Conversión de errores.
- Claves de idempotencia.
- Reintentos seguros.

Los componentes Svelte no deberían construir manualmente URLs ni repetir lógica de autenticación en cada pantalla.

src/types/

Contiene tipos TypeScript utilizados por el frontend.

Puede incluir:

- DTOs de respuestas.
- DTOs de solicitudes.
- Tipos de paginación.
- Tipos de error de API.
- Tipos de usuario y permisos visibles.

Estos tipos representan contratos externos, no entidades internas de dominio Go. Deben cambiar cuando cambie el contrato HTTP, no cuando cambie una tabla interna.

src/lib/

Contiene utilidades generales del frontend.

Ejemplos:

- Formateo de moneda DOP para mostrar.
- Formateo de fechas.
- Validaciones de campos simples.
- Helpers de accesibilidad.
- Constantes visuales.
- Utilidades de navegación.

El formateo de dinero en esta carpeta es visual. El cálculo financiero definitivo sigue siendo responsabilidad del backend.

9.2 client/app.go

Es el punto de integración Go de Wails.

Puede encargarse de:

- Crear la aplicación Wails.
- Exponer métodos necesarios al frontend.
- Administrar el ciclo de vida de la aplicación.
- Integrar capacidades locales permitidas.
- Conectar el frontend con `api_client.go`.

No debe convertirse en una segunda API ni contener reglas de ventas, caja o inventario. El cliente Go local no debe tener credenciales de MySQL.

9.3 client/api_client.go

Implementa el cliente local que se comunica con la API central mediante HTTPS y JSON.

Puede encargarse de:

- Construir solicitudes HTTP.
- Adjuntar la sesión.
- Configurar timeouts.
- Interpretar respuestas.
- Convertir errores técnicos.

- Generar claves de idempotencia.
- Exponer métodos que Wails pueda llamar desde Svelte.

Su función es transportar solicitudes. Si la API responde que una empleada no puede aprobar un ajuste, el cliente debe mostrar el error, no intentar saltarse la restricción.

9.4 client/wails.json

Es la configuración del proyecto Wails.

Puede definir:

- Nombre de la aplicación.
- Identidad y versión del producto.
- Directorio del frontend.
- Comandos de desarrollo y compilación.
- Configuración de empaquetado.
- Iconos y metadatos.
- Opciones específicas de Wails.

No debe utilizarse para colocar secretos ni credenciales de servidores.

10. Flujo completo de una venta

La razón de toda esta separación se aprecia mejor en un flujo concreto.

Plain Text

1. Svelte captura productos y pagos.
2. El frontend solicita completar la venta.
3. Wails llama al cliente Go local.
4. El cliente Go envía HTTPS + JSON a la API.
5. El handler HTTP convierte JSON en un comando.
6. El caso de uso valida usuario, caja, precios, descuentos y crédito.
7. El dominio calcula y valida la venta.
8. Los puertos coordinan inventario, caja, crédito, facturación y auditoría.
9. La infraestructura ejecuta SQL dentro de una transacción MySQL.
10. La API devuelve un DTO de resultado.
11. Wails entrega la respuesta a Svelte.
12. Svelte actualiza la pantalla y muestra el comprobante.

En ningún momento Svelte accede a MySQL, y tampoco puede convertir una venta inválida en válida simplemente modificando el JavaScript de la aplicación.

11. Flujo de dependencias

La dirección recomendada es:

Plain Text

```
Svelte
  ↓
Wails / cliente Go local
  ↓ HTTPS + JSON
HTTP adapter de la API
  ↓
Caso de uso de application
  ↓
Entidades y reglas de domain
  ↑
Puertos implementados por infrastructure
  ↓
MySQL / correo / backup / servicios externos
```

La flecha conceptual de dependencia apunta hacia el centro. El dominio no debe importar `net/http` , `database/sql` , `sqlc` , Svelte ni Wails.

Regla práctica para decidir dónde colocar código

Pregunta	Lugar probable
¿Es una regla sobre una venta, pago o devolución?	<code>modules/<modulo>/domain/</code>
¿Coordina varios objetos o repositorios para realizar una acción?	<code>modules/<modulo>/application/commands/</code>
¿Es una consulta de lectura o reporte?	<code>modules/<modulo>/application/queries/</code>
¿Convierte HTTP/JSON a una solicitud interna?	<code>modules/<modulo>/adapters/http/</code>
¿Implementa una interfaz usando MySQL?	<code>modules/<modulo>/infrastructure/mysql/</code>
¿Es conexión, logging, reloj, correo o configuración técnica?	<code>internal/platform/</code>

¿Es un tipo verdaderamente común de negocio?	internal/shared/
¿Es un componente visual reutilizable?	client/frontend/src/components/
¿Es una pantalla o comportamiento de una capacidad?	client/frontend/src/features/
¿Es un contrato visible para el cliente?	backend/contracts/http/
¿Es SQL versionado o código generado?	backend/db/

12. Resumen final

La estructura separa tres preocupaciones que no deben mezclarse:

1. **Reglas de negocio:** viven en `internal/modules/*/domain` y en los casos de uso de `application`.
2. **Detalles técnicos del servidor:** viven en `platform`, `infrastructure`, `db` y los adaptadores HTTP.
3. **Interfaz y experiencia de usuario:** viven en `client/frontend`, Wails y el cliente Go local.

El objetivo no es crear muchas carpetas por formalidad. El objetivo es que una modificación en MySQL no obligue a reescribir el dominio, que una nueva interfaz no cambie las reglas de ventas y que una futura integración fiscal o de impresión pueda añadirse mediante un adaptador.

Si una operación protege dinero, inventario, crédito o trazabilidad, debe pasar por un caso de uso del backend y ejecutarse con las validaciones, autorizaciones, auditoría e idempotencia correspondientes.